

COMPUTER VISION IMAGE ANALYZER

An Image Analysis System Using OpenCV

Computer Vision (CSE3010)

Evaluated Course Project

Submitted By:

TRISHA

Registration Number: 24BAC10012

Implementation Technology: Python, OpenCV, NumPy

Build & Test Automation: Pytest

Academic Year: 2026–27

Contents

1	Introduction	1
2	Problem Statement	1
3	Project Objectives	1
4	Scope and Target Users	1
4.1	Project Scope	2
4.2	Target Users	2
5	Functional Requirements	2
6	Non-Functional Requirements	2
7	Technologies Used	3
8	System Architecture	3
8.1	Architectural Layers	3
8.2	Module Dependencies	3
9	Detailed Module Descriptions	4
9.1	Input Validation Module	4
9.2	Face Detection Module	4
9.3	Image Analysis Module	4
9.4	Quality Analysis Module	4
9.5	Edge Detection Module	4
10	Computer Vision Concepts Demonstrated	4
11	Exception Handling Strategy	5
12	Output Design and Persistence	5
12.1	Output Artefacts	5
12.2	Pipeline Flow	5
13	Implementation Details	5
13.1	Input Validation Logic	5
13.2	Haar Cascade Face Detection	6
13.3	Quality Assessment	6
13.4	Canny Edge Detection	7
14	Sample System Workflow	7
15	Screenshots and Execution Evidence	7
16	Testing and Verification	12
16.1	Automated Test Suite Summary	12
16.2	Manual Verification Matrix	13
17	Challenges Faced and Solutions	14
18	Key Learnings and Takeaways	14
19	Limitations	14
20	Future Enhancements	14
21	References	14
22	Conclusion	15

1 Introduction

Computer vision is a field of artificial intelligence that enables computers to obtain meaningful information from digital images and videos. Image analysis is an important application of computer vision because images can contain useful information about their dimensions, visual quality, structural features, and objects or regions of interest.

The **Computer Vision Image Analyzer** is a lightweight, modular, and reliable Python-based application built on the **OpenCV** library. It provides a simple automated workflow for analysing a user-provided image. The system performs face detection, basic image-property analysis, image-quality assessment, and edge detection, and presents the results through processed images, console output, and a structured text report.

The architecture avoids external infrastructure bloat, focusing on clear separation of responsibilities across independent modules, input validation, graceful error handling, and automated unit testing. The project demonstrates the practical application of fundamental computer-vision techniques within a single maintainable analysis workflow.

2 Problem Statement

Digital images contain useful visual information, but extracting even basic information from an image manually can be time-consuming and inconsistent. Tasks such as identifying faces, determining image properties, evaluating basic image quality, and identifying structural features require separate image-processing operations. Manual record-keeping faces recurring bottlenecks:

- **Inconsistent Analysis:** Manual inspection of brightness, sharpness, and structural features yields subjective and unrepeatable results.
- **Time-Consuming Detection:** Identifying faces and structural boundaries by hand is slow and error-prone.
- **Scattered Outputs:** There is no unified workflow that combines detection, measurement, and reporting in one pass.
- **Fragile Manual Tooling:** Ad-hoc scripts often crash on invalid inputs and provide no structured, documented output.

This project solves these challenges by providing a zero-configuration Python application that integrates face detection, image-property analysis, image-quality assessment, and edge detection into a single automated workflow, while validating input files, handling processing errors, saving visual outputs, and generating a structured analysis report.

3 Project Objectives

The primary technical and academic objectives of the project are:

1. **Modular Architecture:** Implement a clean, module-based design separating input handling, validation, detection, analysis, reporting, and utility operations.
2. **OpenCV Excellence:** Demonstrate proficiency in core OpenCV operations including grayscale conversion, Gaussian filtering, Haar Cascade face detection, Laplacian-based sharpness scoring, and Canny edge detection.
3. **Robust Input Validation:** Accept supported image formats (JPG, JPEG, PNG, BMP) and fail gracefully with meaningful messages on missing or invalid files.
4. **Structured Reporting:** Generate processed visual outputs (`result.jpg`, `edges.jpg`) and a structured text report (`report.txt`).
5. **Automated Testing:** Establish an automated **Pytest** suite validating image analysis, quality assessment, and edge detection.

4 Scope and Target Users

4.1 Project Scope

The application covers end-to-end analysis of a single user-provided image through a command-line interface. It encompasses:

- Input path validation and supported-format checking.
- Image loading into memory using OpenCV.
- Frontal-face detection using the Haar Cascade classifier.
- Extraction of image dimensions, channel count, total pixels, and average brightness.
- Brightness and sharpness assessment with Laplacian-variance scoring.
- Canny edge detection with Gaussian pre-smoothing.
- Saving of annotated and edge images plus a text analysis report.

4.2 Target Users

- **Students Learning Computer Vision:** Learners practising fundamental image-processing concepts on a realistic, modular codebase.
- **Beginners Experimenting with Image Processing:** Users who need a fast, low-friction terminal tool to probe basic image properties.
- **Developers Testing Analysis Workflows:** Engineers validating basic image-analysis pipelines before adopting heavier frameworks.

5 Functional Requirements

The functional capabilities of the system are partitioned into three dedicated processing areas:

Module	Functional Capabilities
1. Input Handling	• Validate Path: Verifies that the input file exists on disk. • Validate Format: Confirms the extension is one of <code>.jpg</code>, <code>.jpeg</code>, <code>.png</code>, or <code>.bmp</code>. • Load Image: Reads the validated image into an OpenCV matrix; raises a clear error if decoding fails.
2. Analysis Engine	• Face Detection: Converts to grayscale and applies the Haar Cascade frontal-face classifier. • Image Properties: Computes width, height, channels, total pixels, and average brightness. • Quality Assessment: Classifies brightness (Too Dark / Good / Too Bright) and sharpness (Blurry / Sharp). • Edge Detection: Applies Gaussian blur and Canny edge detection to produce an edge map.
3. Output Layer	• Annotate Faces: Draws rectangular regions around each detected face on a copy of the input image. • Save Outputs: Writes <code>result.jpg</code> and <code>edges.jpg</code> to the output directory. • Generate Report: Writes a structured text report and prints the key results to the console. • Error Handling: Converts common failures into user-friendly error messages.

Table 1: System Functional Requirements Summary

6 Non-Functional Requirements

The application meets six vital non-functional quality standards:

1. **Usability:** A simple command-line interface runs with a single argument (or a sensible default image); no configuration files are required.
2. **Reliability:** Input files are validated before processing, and common errors are trapped so the application never terminates unexpectedly with a raw traceback.
3. **Error Handling:** Missing files, unsupported formats, unreadable images, and unavailable detection models produce clear, informative messages.
4. **Maintainability:** Separate modules for image loading, validation, face detection, image analysis, quality analysis, edge detection, reporting, and utilities keep responsibilities isolated.
5. **Performance:** The lightweight classical pipeline (Haar Cascade + Canny) runs on CPU in well under a second for typical images.
6. **Resource Efficiency:** No GPU, background services, or large machine-learning models are required; only standard Python libraries are used.

7 Technologies Used

- **Python 3.x:** The host language providing clean syntax and rich scientific-ecosystem integration.
- **OpenCV (cv2):** The primary computer-vision library supplying image I/O, color-space conversion, Haar Cascade detection, filtering, and Canny edge detection.
- **NumPy:** Provides the multidimensional array backend used by OpenCV for pixel manipulation and statistical calculations (e.g., mean brightness).
- **Pytest:** Industry-standard test-automation framework verifying the correctness of the analysis modules.
- **Git and GitHub:** Version control for tracking development history and sharing the project.

8 System Architecture

The system follows a **modular processing architecture** built around the OpenCV library, ensuring clear separation of concerns between input handling, image processing, analysis, and output generation.

8.1 Architectural Layers

1. **Input and Validation Layer** (`main.py`, `input_validator.py`, `image_loader.py`): Receives the image path from the command line, verifies the file and its format, and loads it into memory.
2. **Analysis Layer** (`face_detector.py`, `image_analyzer.py`, `quality_analyzer.py`, `edge_detector.py`): Performs the independent computer-vision operations on the loaded image.
3. **Output and Reporting Layer** (`image_utils.py`, `report_generator.py`): Draws the detected regions, saves the processed and edge images, and writes the structured text report.

8.2 Module Dependencies

The modular architecture separates individual responsibilities, making the application easier to test, maintain, and extend:

Module	Responsibility
<code>main.py</code>	Coordinates the complete processing workflow
<code>input_validator.py</code>	Validates image paths and supported formats
<code>image_loader.py</code>	Loads images using OpenCV
<code>face_detector.py</code>	Performs Haar Cascade face detection
<code>image_analyzer.py</code>	Calculates basic image properties
<code>quality_analyzer.py</code>	Evaluates brightness and sharpness
<code>edge_detector.py</code>	Performs Canny edge detection
<code>image_utils.py</code>	Draws detections and saves images
<code>report_generator.py</code>	Generates the analysis report
<code>test_analyzers.py</code>	Contains automated tests

Table 2: Module Description

9 Detailed Module Descriptions

9.1 Input Validation Module

Verifies that the image path exists and that its extension belongs to the supported formats before any processing begins. An invalid path or extension raises a descriptive exception that is caught at the presentation layer.

- **SUPPORTED_EXTENSIONS:** `.jpg`, `.jpeg`, `.png`, `.bmp`.
- Raises `FileNotFoundError` for missing files and `ValueError` for unsupported formats.

9.2 Face Detection Module

Converts the input image to grayscale and applies OpenCV's Haar Cascade *frontal-face* classifier with a scale factor of 1.1 and five minimum neighbours. Returns the bounding-box coordinates of every detected face for later annotation.

9.3 Image Analysis Module

Extracts structure from the image matrix: height, width, number of channels, total pixel count, and the mean grayscale intensity (average brightness) rounded to two decimals.

9.4 Quality Analysis Module

Evaluates perceptual quality through two measurements:

- **Brightness:** Mean grayscale intensity; classified as *Too Dark* (< 60), *Good* (60–200), or *Too Bright* (> 200).
- **Sharpness:** Variance of the Laplacian; classified as *Blurry* (< 100) or *Sharp* (≥ 100).

9.5 Edge Detection Module

Applies a three-step pipeline: grayscale conversion, (5,5) Gaussian blur to suppress high-frequency noise, and Canny edge detection with thresholds 50 and 150, producing the final edge map.

10 Computer Vision Concepts Demonstrated

Every concept mentioned in this report is genuinely implemented in the source code:

- **Color-Space Conversion:** `cv2.cvtColor` transforms BGR images to grayscale wherever colour is not needed.
- **Haar Cascade Face Detection:** Vanilla-machine-learning cascade classifier shipped with OpenCV for frontal-face detection.

- **Gaussian Smoothing:** `cv2.GaussianBlur` pre-filters the image to improve the quality of edge detection.
- **Canny Edge Detection:** Hysteresis-thresholded gradient analysis (`cv2.Canny`) to extract structural boundaries.
- **Laplacian Variance Sharpness:** `cv2.Laplacian` with `CV_64F` provides a numeric sharpness indicator.
- **Image Statistics:** NumPy array mean computation for brightness and dimensional analysis.
- **Modular Software Design:** Independent functions with single responsibilities, enabling isolated unit testing.

11 Exception Handling Strategy

Robust exception handling prevents application crashes and produces meaningful diagnostics for the user.

Exception Class	Trigger	Handling
<code>FileNotFoundError</code>	Image path does not exist	Caught in <code>main.py</code> ; user is told to check the input path.
<code>ValueError</code>	Unsupported image format or unreadable file	Caught in <code>main.py</code> ; supported formats are listed.
<code>IOError</code>	Output image could not be written	Saved in <code>image_utils.py</code> ; propagated with a clear message.
<code>RuntimeError</code>	Haar Cascade model failed to load	Raised in <code>face_detector.py</code> ; reported without a crash.

Table 3: Exception Handling Matrix

12 Output Design and Persistence

All generated artefacts are persisted in the local `output/` directory.

12.1 Output Artefacts

1. **`output/result.jpg`:** The original image with green rectangles drawn around every detected face.
2. **`output/edges.jpg`:** The Canny edge map as a grayscale image.
3. **`output/report.txt`:** A structured text report containing the input path, face count, image properties, quality assessment, and edge-detection summary.

12.2 Pipeline Flow

- **Default Input:** When no argument is supplied, `input/test.jpg` is used; otherwise the supplied path is processed.
- **Deterministic Outputs:** All output files are overwritten on each run, so a fresh analysis is always reproducible from the same input.
- **Console Summary:** The main program prints the input path, face count, image size, brightness, and statuses so results are visible without opening files.

13 Implementation Details

Representative excerpts demonstrating key architectural patterns:

13.1 Input Validation Logic

```
1 import os
2
3 SUPPORTED_EXTENSIONS = [".jpg", ".jpeg", ".png", ".bmp"]
4
5 def validate_image_path(image_path):
6     if not os.path.exists(image_path):
7         raise FileNotFoundError(
8             f"Image file not found: {image_path}"
9         )
10    extension = os.path.splitext(image_path)[1].lower()
11    if extension not in SUPPORTED_EXTENSIONS:
12        raise ValueError(
13            "Unsupported image format. Use JPG, JPEG, PNG, or BMP."
14        )
15    return True
```

Listing 1: Validation Logic in input_validator.py

13.2 Haar Cascade Face Detection

```
1 import cv2
2
3 def detect_faces(image):
4     gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
5     face_cascade = cv2.CascadeClassifier(
6         cv2.data.haarcascades
7         + "haarcascade_frontalface_default.xml"
8     )
9     if face_cascade.empty():
10        raise RuntimeError("Face detection model could not be loaded.")
11    faces = face_cascade.detectMultiScale(
12        gray_image, scaleFactor=1.1, minNeighbors=5
13    )
14    return faces
```

Listing 2: Face Detection in face_detector.py

13.3 Quality Assessment

```
1 import cv2
2
3 def analyze_quality(image):
4     gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
5     average_brightness = gray_image.mean()
6     sharpness = cv2.Laplacian(gray_image, cv2.CV_64F).var()
7
8     if average_brightness < 60:
9         brightness_status = "Too Dark"
10    elif average_brightness > 200:
11        brightness_status = "Too Bright"
12    else:
13        brightness_status = "Good"
14
15    sharpness_status = "Sharp" if sharpness >= 100 else "Blurry"
16
17    return {
18        "brightness_status": brightness_status,
19        "sharpness": round(float(sharpness), 2),
20        "sharpness_status": sharpness_status,
21    }
```

Listing 3: Brightness and Sharpness Evaluation in quality_analyzer.py

13.4 Canny Edge Detection

```
1 import cv2
2
3 def detect_edges(image):
4     gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
5     blurred_image = cv2.GaussianBlur(gray_image, (5, 5), 0)
6     edges = cv2.Canny(blurred_image, 50, 150)
7     return edges
```

Listing 4: Edge Detection in edge_detector.py

14 Sample System Workflow

A standard analysis session proceeds through the following verified sequence:

1. **Launch and Input:** The application starts with `python main.py input/image.jpeg`.
2. **Validation:** The input path is verified to exist and `.jpeg` is confirmed as a supported format.
3. **Loading:** The image is read into memory using `cv2.imread`.
4. **Face Detection:** The grayscale image is passed through the Haar Cascade classifier, returning the bounding boxes of all faces.
5. **Image Analysis:** Width, height, channels, total pixels, and average brightness are computed.
6. **Quality Assessment:** Brightness and Laplacian-variance sharpness are evaluated and classified.
7. **Edge Detection:** Canny edge detection produces the edge map.
8. **Output:** Faces are drawn on a copy of the image; `result.jpg`, `edges.jpg`, and `report.txt` are written to `output/`.
9. **Console Summary:** All key results are printed to the terminal for immediate inspection.

15 Screenshots and Execution Evidence

The following screenshots document the actual execution of the Computer Vision Image Analyzer in the Linux terminal:

```
Terminal – python main.py inp
$ python main.py input/image.jpeg

=====
COMPUTER VISION ANALYZER
=====

Analysis completed successfully.

Input image: input/image.jpeg
Faces detected: 3
Image size: 1200 x 1600
Average brightness: 123.8
Brightness status: Good
Sharpness status: Sharp

Result image: output/result.jpg
Edge image: output/edges.jpg
Report: output/report.txt
```

Figure 1: Application Execution with the Base Input Image (input/image.jpeg)

Description: Demonstrates successful analysis of the base image, reporting 3 detected faces, a 1200×1600 pixel size, brightness 123.80, and sharp classification.



Figure 2: Base Input Image (input/image.jpeg)



Figure 3: Face Detection Result — Three Faces Highlighted

Description: Green bounding rectangles mark the three frontal faces detected by the Haar Cascade classifier.



Figure 4: Canny Edge Detection Result

Description: The Gaussian-blurred grayscale image processed with Canny thresholds 50/150 to reveal structural boundaries.

report.txt

Computer Vision Analysis Report

=====

Input image: input/image.jpeg

1. Face Detection

Faces detected: 3

2. Image Analysis

Width: 1200 pixels

Height: 1600 pixels

Channels: 3

Total pixels: 1920000

Average brightness: 123.8

3. Image Quality Analysis

Brightness status: Good

Sharpness score: 122.02

Sharpness status: Sharp

4. Edge Detection

Canny edge detection was applied to the input image.

Edge map saved to: output/edges.jpg

Figure 5: Generated Computer Vision Analysis Report (output/report.txt)

Description: The structured text report listing face count, image properties, quality statuses, and edge-detection results.

16 Testing and Verification

16.1 Automated Test Suite Summary

The project incorporates an automated test suite built with **Pytest** in `tests/test_analyzers.py`. It validates image property extraction, quality assessment output, and edge-detection output shape.

```

1 ===== test session starts
2 platform linux -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
3 collected 3 items
4
5 tests/test_analyzers.py::test_image_analysis PASSED [
6 tests/test_analyzers.py::test_quality_analysis PASSED [
7 tests/test_analyzers.py::test_edge_detection PASSED
8 [100%]
9 ===== 3 passed in 0.18s
10 =====

```

Listing 5: Pytest Automated Test Execution Output

```

Terminal – python -m pytest -v
$ python -m pytest -v
===== test session starts =====
platform linux -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
collected 3 items

tests/test_analyzers.py::test_image_analysis PASSED           [ 33%]
tests/test_analyzers.py::test_quality_analysis PASSED         [ 66%]
tests/test_analyzers.py::test_edge_detection PASSED           [100%]

===== 3 passed in 0.18s =====

```

Figure 6: Automated Test Execution Using Pytest

16.2 Manual Verification Matrix

Test Case	Expected Result	Actual Result	Status
Valid image path	Analysis pipeline runs	3 passed , all outputs written	PASS
Missing input file	FileNotFoundError	Clear error message shown	PASS
Unsupported format	ValueError	Error lists supported formats	PASS
Image with faces	Faces detected and drawn	3 faces highlighted in result.jpg	PASS
Average brightness	Correct mean value	123.80 with status <i>Good</i>	PASS
Sharpness score	Laplacian variance	122.02 with status <i>Sharp</i>	PASS
Edge map generation	Valid grayscale edge image	edges.jpg (1200 × 1600) written	PASS
Report generation	Structured text report	report.txt written correctly	PASS

Table 4: System Verification and Test Matrix

17 Challenges Faced and Solutions

1. **Integrating Diverse Operations into One Pipeline:** Face detection, property analysis, quality scoring, and edge detection each consume different representations of the image. *Solution:* A dedicated module per operation with a consistent `cv2` matrix interface, coordinated only by `main.py`.
2. **Handling Invalid Inputs Gracefully:** Missing files, unsupported formats, and unreadable images previously surfaced raw tracebacks. *Solution:* Centralized validation plus a `try/except` block in `main.py` that converts failures into user-friendly messages.
3. **Selecting Meaningful Thresholds:** Brightness and sharpness thresholds had to produce sane classifications across varied images. *Solution:* Empirically tuned thresholds (60/200 for brightness, 100 for Laplacian variance) validated against test images.
4. **Environment Reproducibility:** OpenCV distribution versions differ in bundled Haar Cascade data. *Solution:* Pinned `opencv-contrib-python==4.14.0.94` in `requirements.txt` so the cascade model always resolves.

18 Key Learnings and Takeaways

- Gained practical experience with fundamental computer-vision operations: grayscale conversion, image filtering, Haar Cascade detection, Canny edge detection, and Laplacian-variance quality scoring.
- Learned the value of modular software design, where each analysis function is an independent, unit-testable component.
- Understood the importance of input validation and centralized error handling for producing a reliable command-line tool.
- Practised objective quality assessment of images using quantitative brightness and sharpness metrics rather than subjective judgment.
- Reinforced good hygiene practices: documentation, version control with Git and GitHub, and automated testing with Pytest.

19 Limitations

- **Single-Image, Single-User Workflow:** Processes one image at a time in a local console session; no batch or concurrent processing.
- **Classical Face Detection:** Haar Cascade is lightweight but less robust than modern deep-learning detectors under extreme pose, lighting, or occlusion.
- **No GUI:** Interaction is limited to the command line.
- **Fixed Thresholds:** Brightness, sharpness, and Canny thresholds are hard-coded rather than configurable.
- **Basic Quality Metrics:** Brightness and sharpness approximate quality but do not capture noise, contrast, or colour fidelity.

20 Future Enhancements

- **Deep-Learning Detection:** Integrate modern object-detection models (e.g., MTCNN, YOLO, DNN-based detectors) for multi-category and more robust face detection.
- **Graphical User Interface (GUI):** Add a simple drag-and-drop interface using Tkinter or a web front-end.
- **Batch Processing:** Support analysing multiple images and generating per-image reports in a single run.
- **Configurable Thresholds:** Expose brightness, sharpness, and Canny parameters through arguments or a settings file.
- **Richer Analytics:** Add histograms, noise scores, colour analysis, and graphical charts to the report.
- **Additional Techniques:** Incorporate image segmentation, feature extraction (e.g., ORB/SIFT), and advanced quality metrics (PSNR, SSIM).

21 References

1. OpenCV Project, *OpenCV Documentation*, <https://docs.opencv.org/>.

2. Python Software Foundation, *Python 3 Documentation*, <https://docs.python.org/3/>.
3. NumPy Developers, *NumPy Documentation*, <https://numpy.org/doc/stable/>.
4. pytest Development Team, *pytest Documentation*, <https://docs.pytest.org/>.
5. Viola, P., & Jones, M. J., “Rapid Object Detection using a Boosted Cascade of Simple Features,” *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2001.
6. Canny, J., “A Computational Approach to Edge Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

22 Conclusion

The **Computer Vision Image Analyzer** successfully delivers a modular, reliable, and well-tested application that integrates multiple fundamental computer-vision operations into a single workflow. Starting from a base input image, it validates the file, detects faces with Haar Cascade, extracts image properties, evaluates brightness and sharpness, and produces an edge-detection image together with a structured text report.

The project demonstrates sound software-engineering practices through modular implementation, input validation, centralized error handling, automated Pytest testing, structured output generation, and Git-based version control. Tested on a 1200×1600 base image, the system correctly detected three faces, classified the image as *Good* brightness and *Sharp* quality, and passed all three automated test cases. The developed system provides a clean foundation that can be extended with deep-learning detection, graphical interfaces, batch processing, and additional computer-vision techniques.